

GLM

Bhavani Chalasani - 2022101014

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.1      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(stringr)
library(lubridate)
library(car)
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
##     recode
##
## The following object is masked from 'package:purrr':
##
##     some
```

```
library(nortest)
library(effects)
```

```
## Registered S3 method overwritten by 'lme4':
##   method      from
##   na.action.merMod car
## lattice theme set by effectsTheme()
## See ?effectsTheme for details.
```

```
library(e1071)
```

```
##
```

```

## Attaching package: 'e1071'
##
## The following object is masked from 'package:ggplot2':
##
##      element

library(marginaleffects)
library(betareg)

get_time_of_day <- function(ts) {
  time_ist <- as_datetime(ts / 1000, tz = "Asia/Kolkata")
  hour <- hour(time_ist)
  case_when(
    hour >= 6 & hour < 16 ~ "Day time",
    # hour >= 12 & hour < 17 ~ "Afternoon",
    hour >= 16 & hour < 20 ~ "Evening",
    TRUE ~ "Night"
  )
}

raw_dataset <- read_csv(
  "sentence_memorability_data.csv",
  na = c("", "NA", "N/A"),
  show_col_types = FALSE
)

data <- raw_dataset %>%
  arrange(participant_ID, Timestamp) %>%
  mutate(
    Accuracy_IR = suppressWarnings(as.numeric(Accuracy_IR)),
    Accuracy_WR = suppressWarnings(as.numeric(Accuracy_WR)),
    prev_IR = lag(Accuracy_IR)
  ) %>%
  group_by(participant_ID) %>%
  summarise(
    Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
    Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown", na.rm = TRUE),
    Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy_IR == 1, na.rm = TRUE),
    False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

    Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
    False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
    IR_Memorability = Hit_Rate - False_Alarm_Rate,

    WR_correct = sum(Event == "WR pressed" & Accuracy_WR == 1 & prev_IR == 1, na.rm = TRUE),
    WR_total = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),
    WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),

    IR_Presses = sum(Event == "IR pressed", na.rm = TRUE),
    WR_Presses = sum(Event == "WR pressed", na.rm = TRUE),
    avg_RT_IR = mean(Reaction_time_IR, na.rm = TRUE),
    avg_RT_WR = mean(Reaction_time_WR, na.rm = TRUE),
    avg_timestamp = mean(Timestamp, na.rm = TRUE),

```

```

    .groups = "drop"
  ) %>%
  mutate(
    Time_of_Day = get_time_of_day(avg_timestamp)
  )

data <- data %>%
  mutate(
    Time_of_Day = factor(Time_of_Day,
                        levels = c("Day time", "Evening", "Night"))
  )

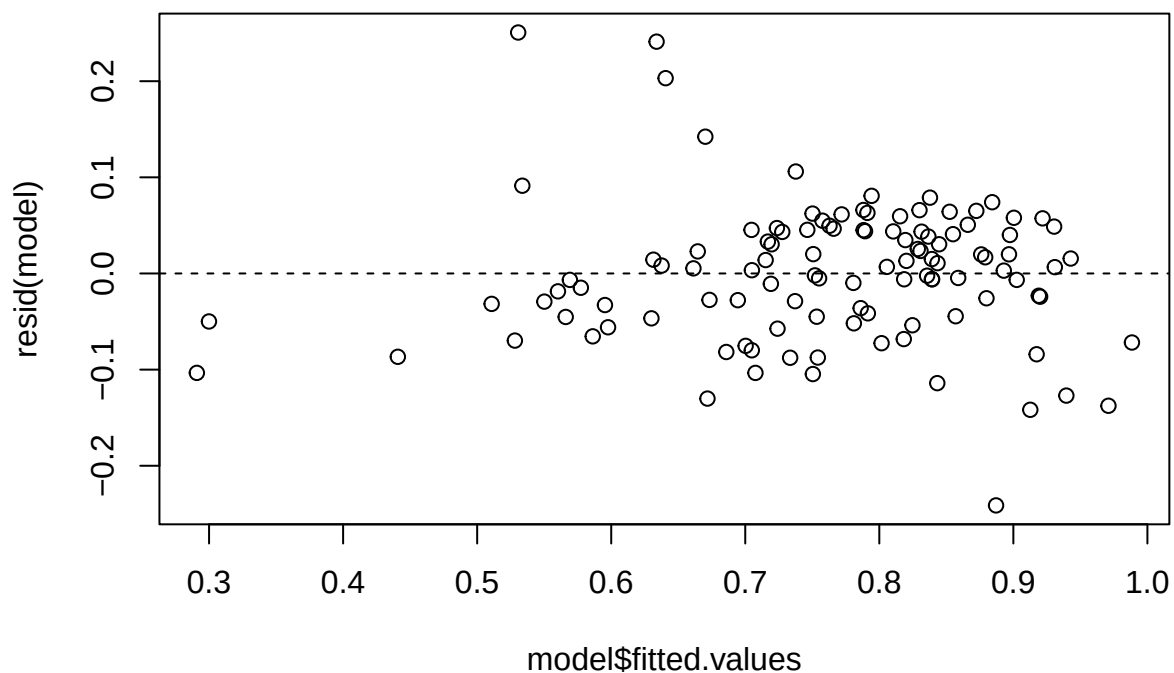
# Testing assumptions - IR memorability

model <- lm(IR_Memorability ~ avg_RT_IR + IR_Presses + Time_of_Day, data = data)

# Linearity and homoskedasticity

plot(model$fitted.values, resid(model))
abline(h = 0, lty = 2)

```



```

ncvTest(model)

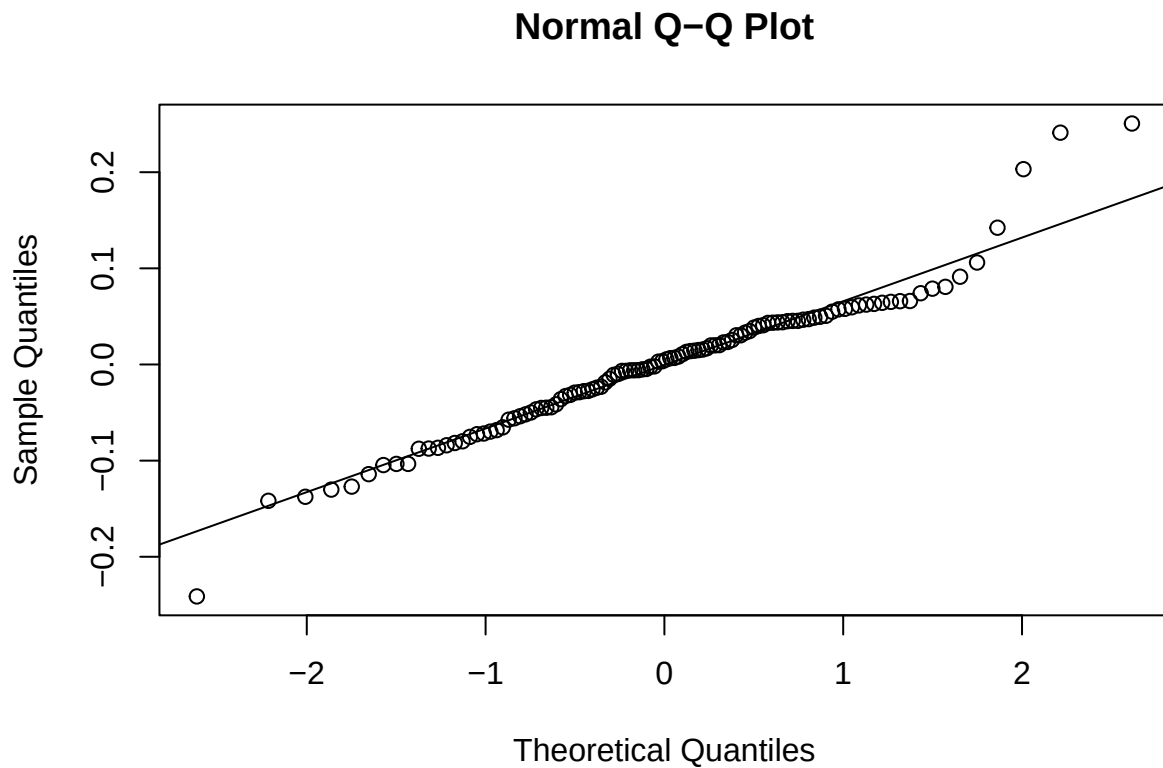
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values

```

```
## Chisquare = 3.689357, Df = 1, p = 0.054761
```

```
# Normality
```

```
qqnorm(resid(model))  
qqline(resid(model))
```

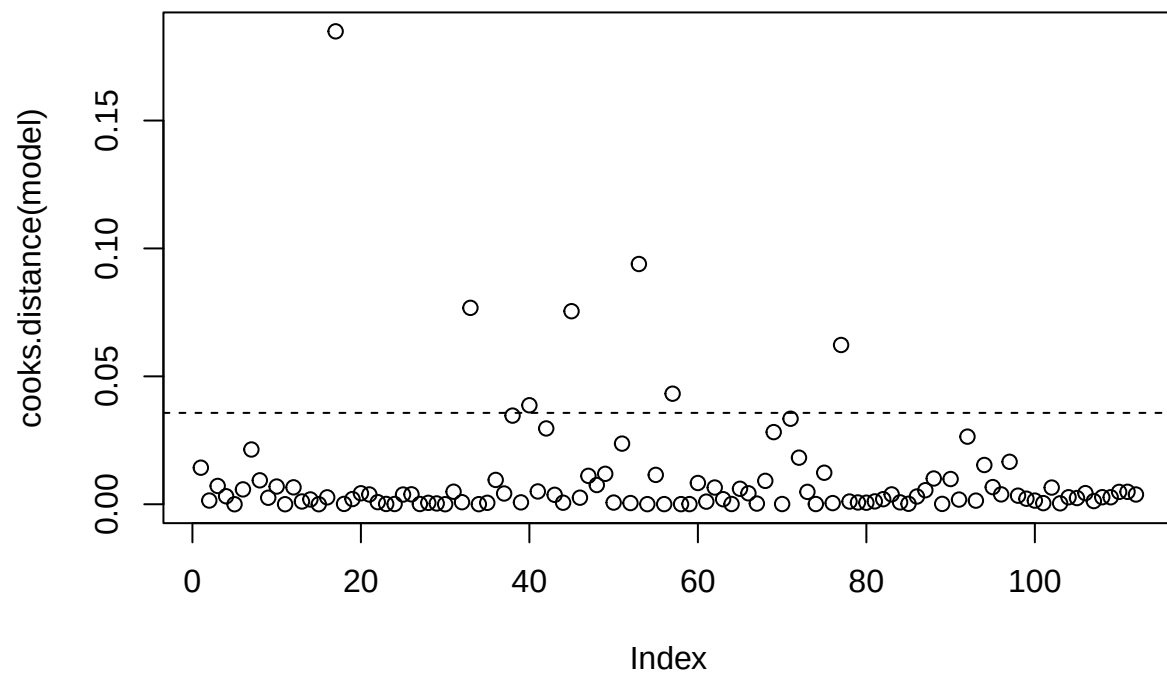


```
residuals_model <- residuals(model)  
ad.test(residuals_model)
```

```
##  
## Anderson-Darling normality test  
##  
## data: residuals_model  
## A = 1.0519, p-value = 0.008809
```

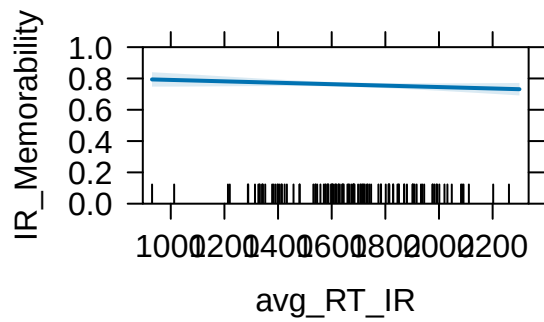
```
# Outliers
```

```
plot(cooks.distance(model))  
abline(h = 4/nrow(data), lty = 2)
```

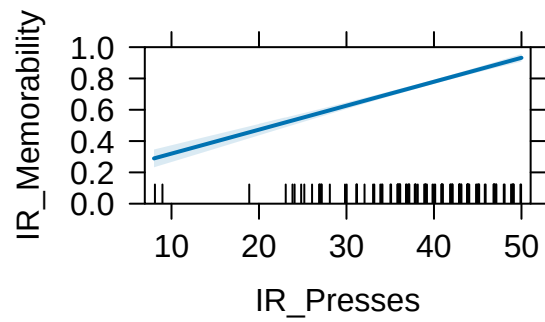


```
# Transforming the data because the GLM assumptions were violated  
plot(allEffects(model), ylim = c(0, 1))
```

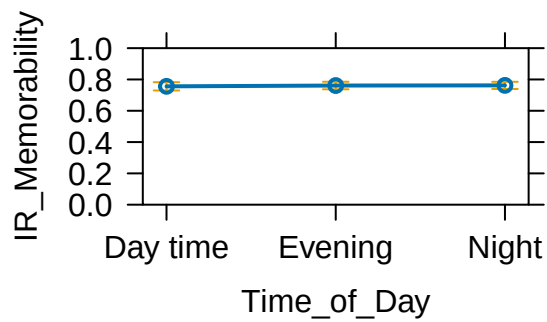
avg_RT_IR effect plot



IR_Presses effect plot

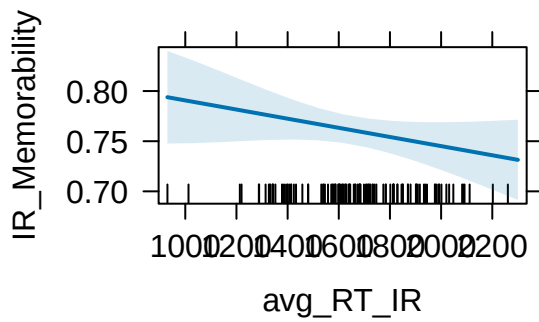


Time_of_Day effect plot

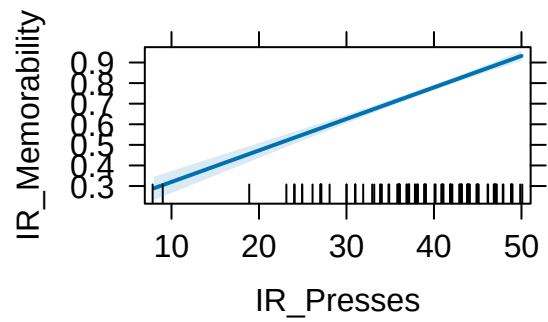


```
plot(allEffects(model))
```

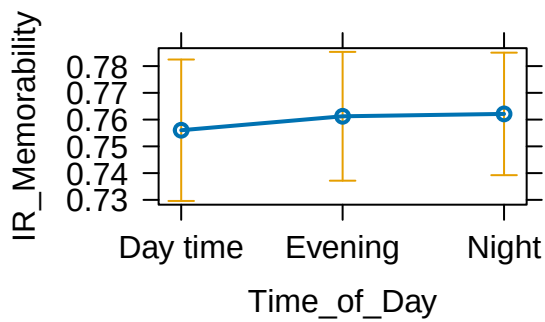
avg_RT_IR effect plot



IR_Presses effect plot

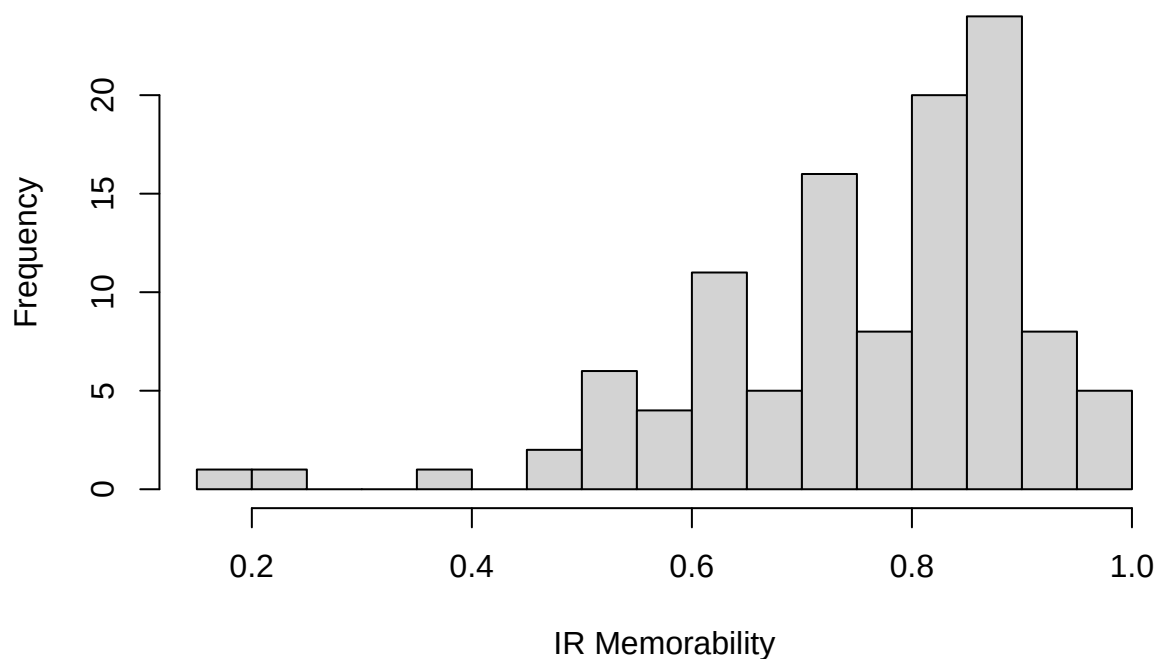


Time_of_Day effect plot



```
hist(data$IR_Memorability,  
      breaks = 20,  
      main = "Distribution of IR Memorability",  
      xlab = "IR Memorability")
```

Distribution of IR Memorability



```
# Fixing linearity
data <- data %>%
  mutate(
    log_RT_IR = log(avg_RT_IR)
  )

# Fixing homoskedasticity and normality - checking skewness to see which transform to apply
cat("Checking skew:\n")
```

```
## Checking skew:
```

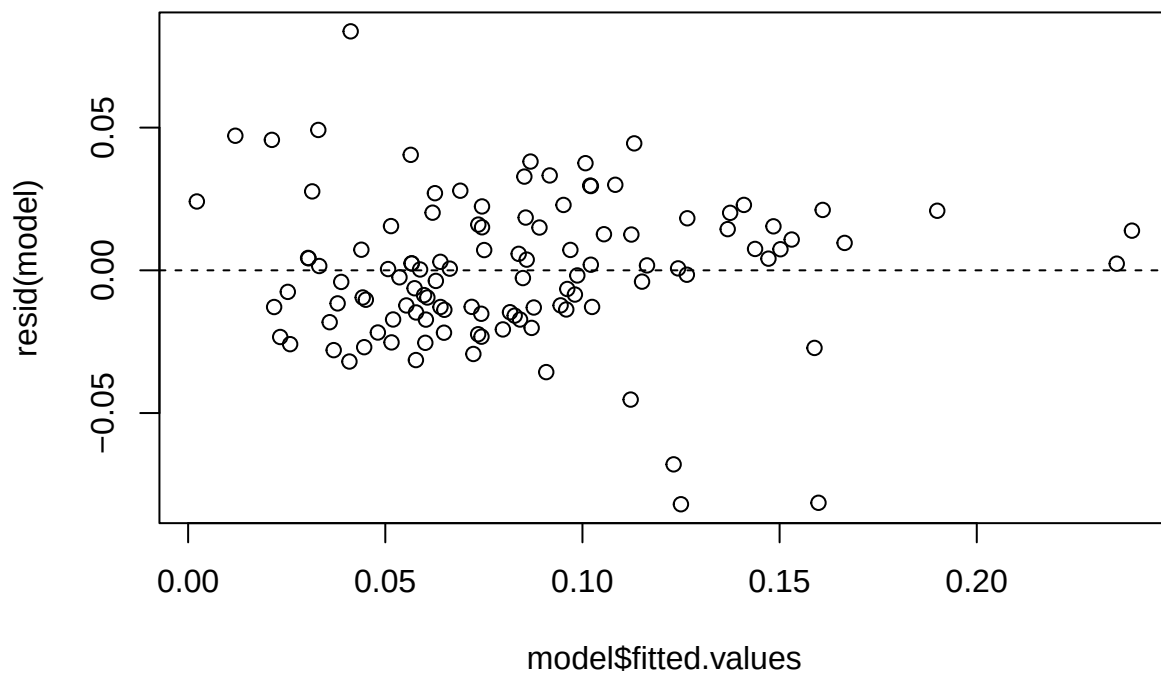
```
skewness(residuals_model)
```

```
## [1] 0.2991809
```

```
K <- max(data$IR_Memorability, na.rm = TRUE) + 1
data <- data %>%
  mutate(
    IR_transformed = log10(K - IR_Memorability)
  )

model <- lm(IR_transformed ~ log_RT_IR + IR_Presses + Time_of_Day, data = data)

plot(model$fitted.values, resid(model))
abline(h = 0, lty = 2)
```

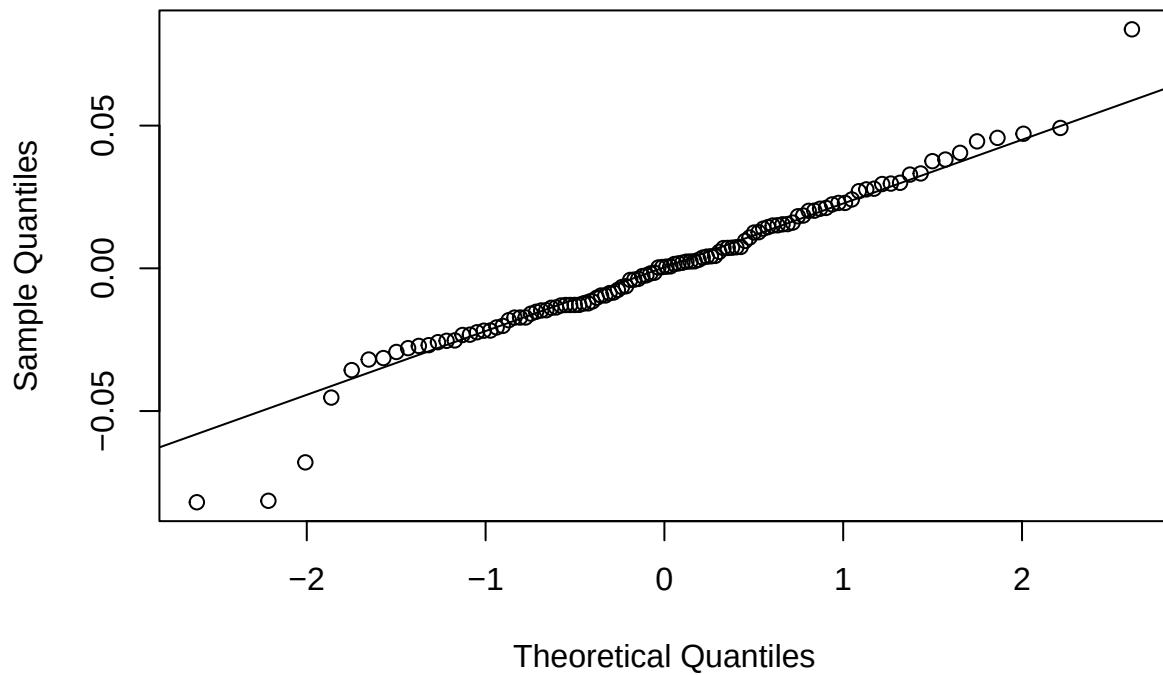
```
ncvTest(model)
```

```
## Non-constant Variance Score Test  
## Variance formula: ~ fitted.values  
## Chisquare = 0.4259475, Df = 1, p = 0.51398
```

```
# Normality
```

```
qqnorm(resid(model))  
qqline(resid(model))
```

Normal Q-Q Plot

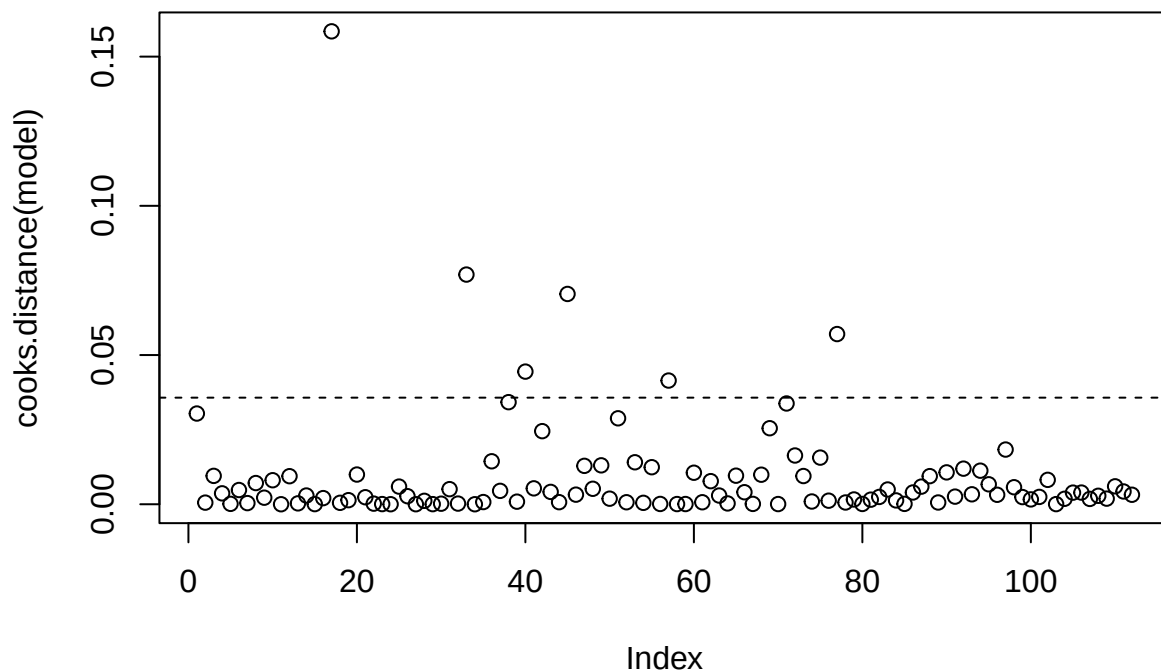


```
residuals_model <- residuals(model)
ad.test(residuals_model)
```

```
##
## Anderson-Darling normality test
##
## data: residuals_model
## A = 0.65947, p-value = 0.083
```

```
# Outliers
```

```
plot(cooks.distance(model))
abline(h = 4/nrow(data), lty = 2)
```



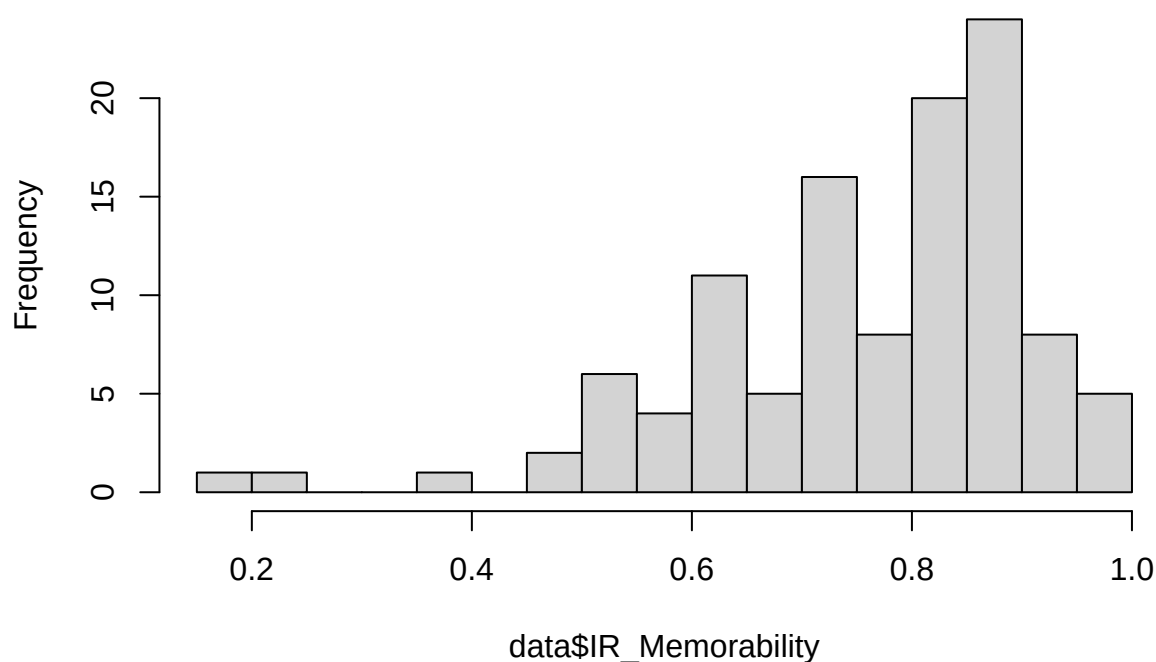
```
# Doing the above with the checks again showed that we cannot transform the data to become normal
```

```
# checking assumptions for beta regression - values should be bw 0 and 1 and acceptable shape - here it  
any(data$IR_Memorability <= 0 | data$IR_Memorability >= 1, na.rm = TRUE)
```

```
## [1] FALSE
```

```
hist(data$IR_Memorability, breaks = 20)
```

Histogram of data\$IR_Memorability



```
model <- betareg(IR_Memorability ~ avg_RT_IR + IR_Presses + Time_of_Day, data = data)

# removing highly correlated predictors using VIF

removed_vars <- c()
repeat {
  vif_values <- vif(model)
  threshold = 5

  if (is.matrix(vif_values)) {
    vif_values <- vif_values[, "GVIF^(1/(2*Df))"]
  }

  max_vif <- max(vif_values)

  if (max_vif <= threshold) break

  variable_to_remove <- names(which.max(vif_values))
  removed_vars <- c(removed_vars, variable_to_remove)
  formula_current <- formula(model)
  predictors <- attr(terms(formula_current), "term.labels")
  predictors <- predictors[predictors != variable_to_remove]
  response <- all.vars(formula_current)[1]
  new_formula <- as.formula(
    paste(response, "~", paste(predictors, collapse = " + "))
  )
}
```

```

model <- betareg(new_formula, data = data)
}

cat("Variables removed due to high VIF for model2:\n")

## Variables removed due to high VIF for model2:

print(removed_vars)

## NULL

# interpreting the model
cat("\nInterpretation\n")

##
## Interpretation

summary(model)

##
## Call:
## betareg(formula = IR_Memorability ~ avg_RT_IR + IR_Presses + Time_of_Day,
## data = data)
##
## Quantile residuals:
##      Min       1Q   Median       3Q      Max
## -2.9820 -0.5738 -0.0470  0.5600  3.4246
##
## Coefficients (mean model with logit link):
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.1025665   0.4130772  -2.669   0.0076 **
## avg_RT_IR      -0.0004402   0.0001778  -2.475   0.0133 *
## IR_Presses      0.0791650   0.0051536  15.361  <2e-16 ***
## Time_of_DayEvening 0.0463689   0.1051576   0.441   0.6593
## Time_of_DayNight 0.0308205   0.1039215   0.297   0.7668
##
## Phi coefficients (precision model with identity link):
##      Estimate Std. Error z value Pr(>|z|)
## (phi)   31.271      4.147   7.541 4.66e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Type of estimator: ML (maximum likelihood)
## Log-likelihood: 143.3 on 6 Df
## Pseudo R-squared: 0.7001
## Number of iterations: 14 (BFGS) + 3 (Fisher scoring)

confint(model)

```

```
##              2.5 %      97.5 %
## (Intercept)    -1.9121829037 -2.929500e-01
## avg_RT_IR      -0.0007887629 -9.160858e-05
## IR_Presses      0.0690641568  8.926580e-02
## Time_of_DayEvening -0.1597360775  2.524740e-01
## Time_of_DayNight -0.1728619712  2.345030e-01
## (phi)          23.1438645314  3.939906e+01
```

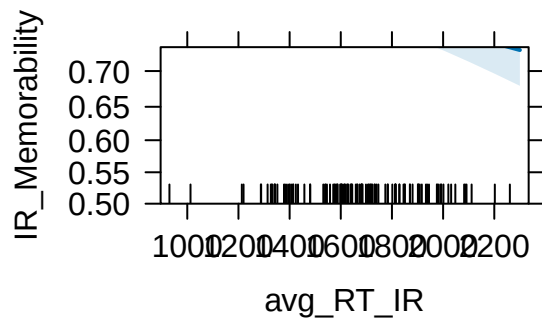
```
avg_slopes(model)
```

```
##
##      Term          Contrast Estimate Std. Error      z Pr(>|z|)      S
## IR_Presses dY/dX          1.31e-02  7.64e-04 17.104   <0.001 215.5
## Time_of_Day Evening - Day time  7.67e-03  1.74e-02  0.440   0.6596   0.6
## Time_of_Day Night - Day time   5.12e-03  1.73e-02  0.296   0.7670   0.4
## avg_RT_IR dY/dX          -7.27e-05  2.93e-05 -2.478   0.0132   6.2
##      2.5 %      97.5 %
##      0.01157  1.46e-02
##     -0.02646  4.18e-02
##     -0.02872  3.90e-02
##     -0.00013 -1.52e-05
##
## Type: response
```

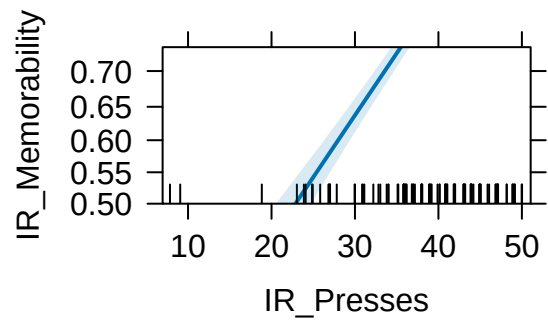
```
# Plotting the model
# The plot below shows how appropriate the model is for our data, and it's quite appropriate
# plot(model)
```

```
plot(allEffects(model), ylim = c(0, 1))
```

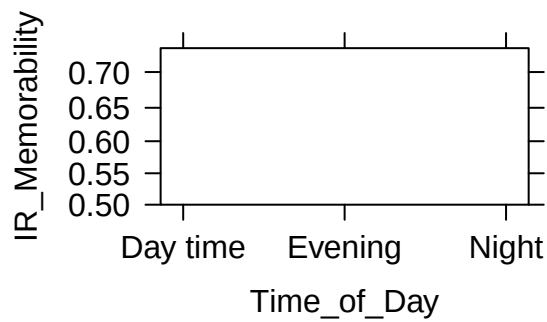
avg_RT_IR effect plot



IR_Presses effect plot

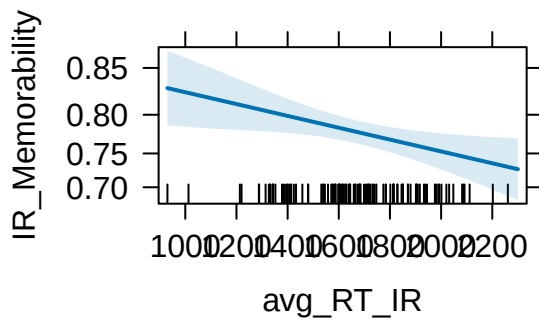


Time_of_Day effect plot

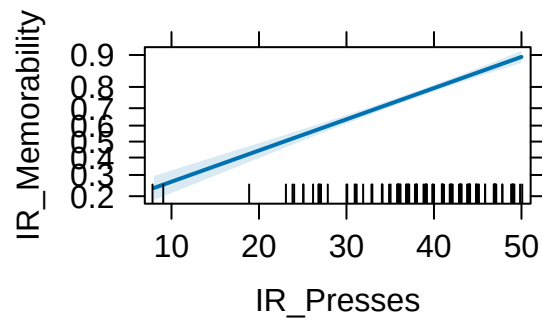


```
plot(allEffects(model))
```

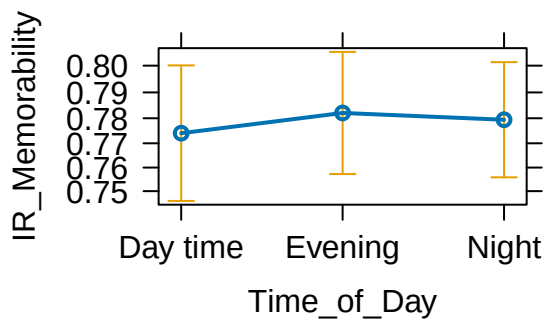
avg_RT_IR effect plot



IR_Presses effect plot



Time_of_Day effect plot



```
cat("AIC value of the model\n")
```

```
## AIC value of the model
```

```
AIC(model)
```

```
## [1] -274.5131
```

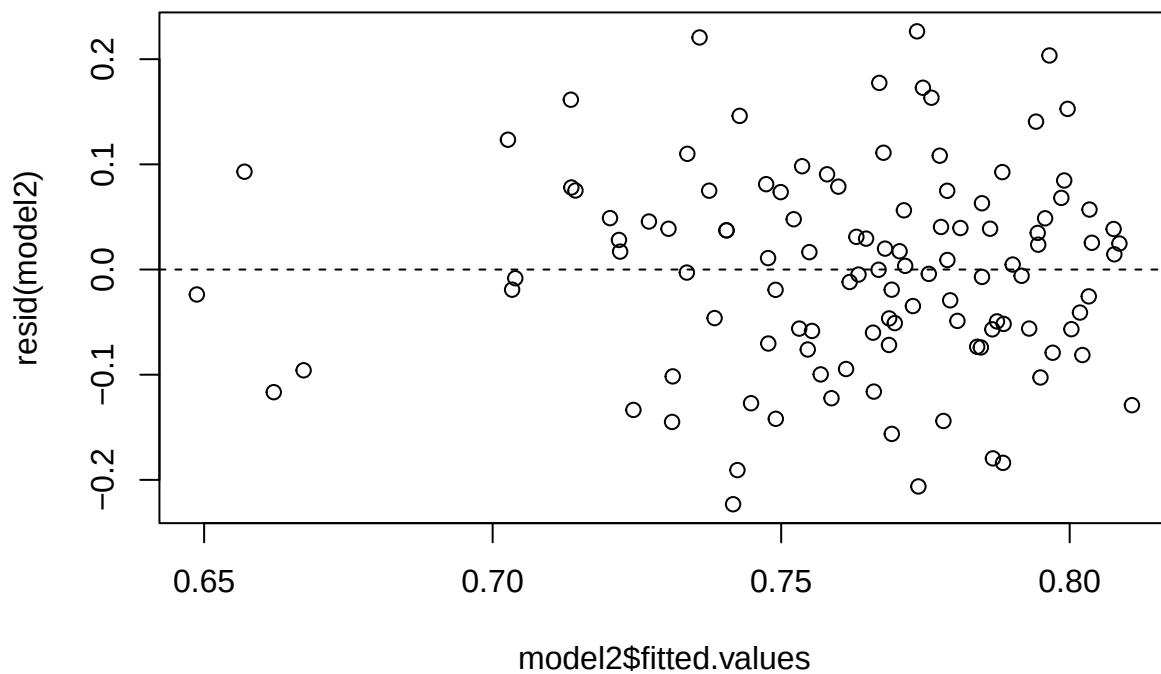
```
# Testing assumptions - WR Accuracy
```

```
model2 <- lm(WR_Accuracy ~ avg_RT_WR + WR_Presses + Time_of_Day, data = data)
```

```
# Linearity and homoskedasticity
```

```
plot(model2$fitted.values, resid(model2))
```

```
abline(h = 0, lty = 2)
```

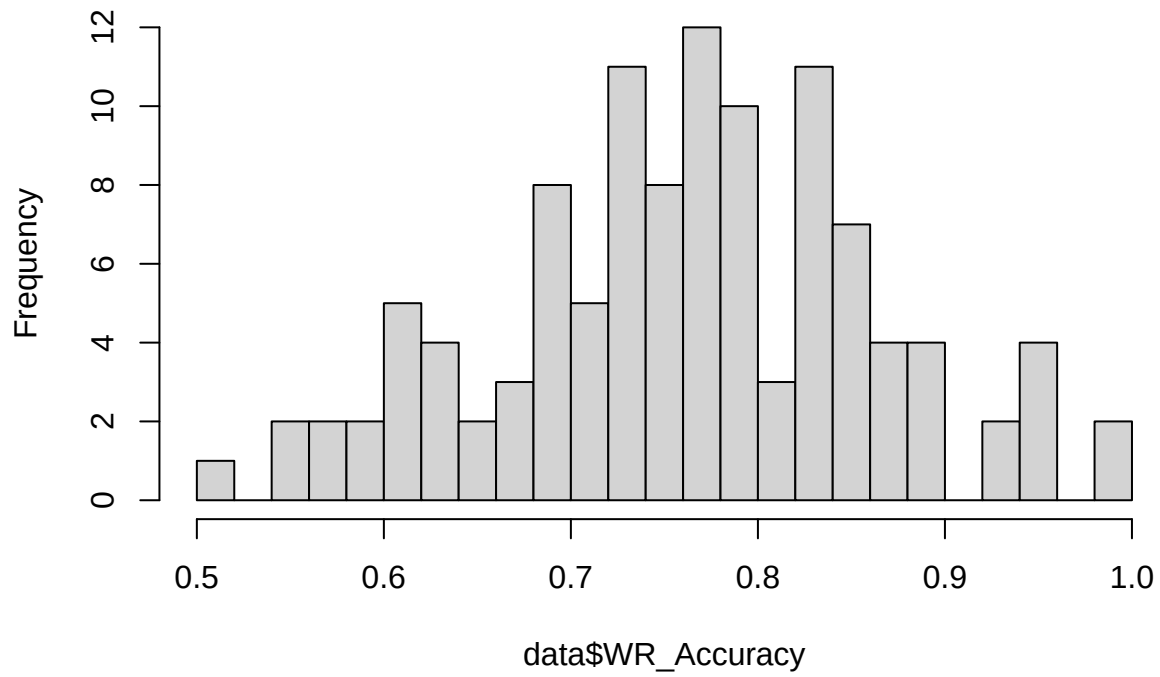



```
ncvTest(model2)
```

```
## Non-constant Variance Score Test  
## Variance formula: ~ fitted.values  
## Chisquare = 0.1648348, Df = 1, p = 0.68474
```

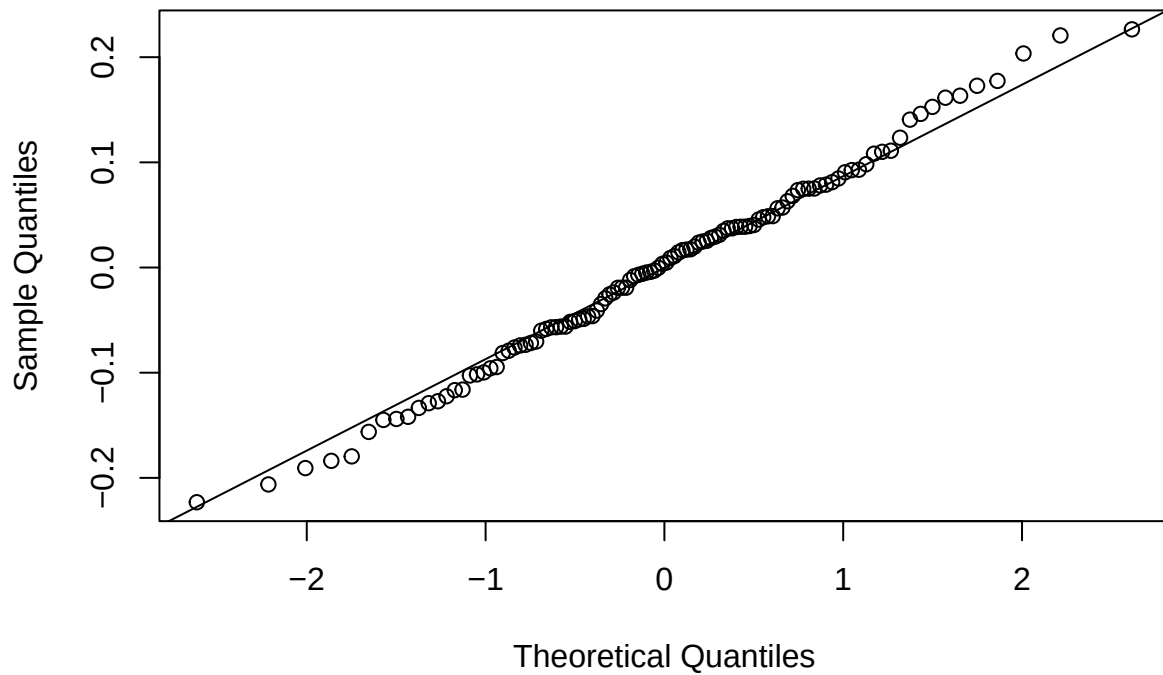
```
# Normality  
hist(data$WR_Accuracy, breaks = 20)
```

Histogram of data\$WR_Accuracy



```
qqnorm(resid(model2))  
qqline(resid(model2))
```

Normal Q-Q Plot

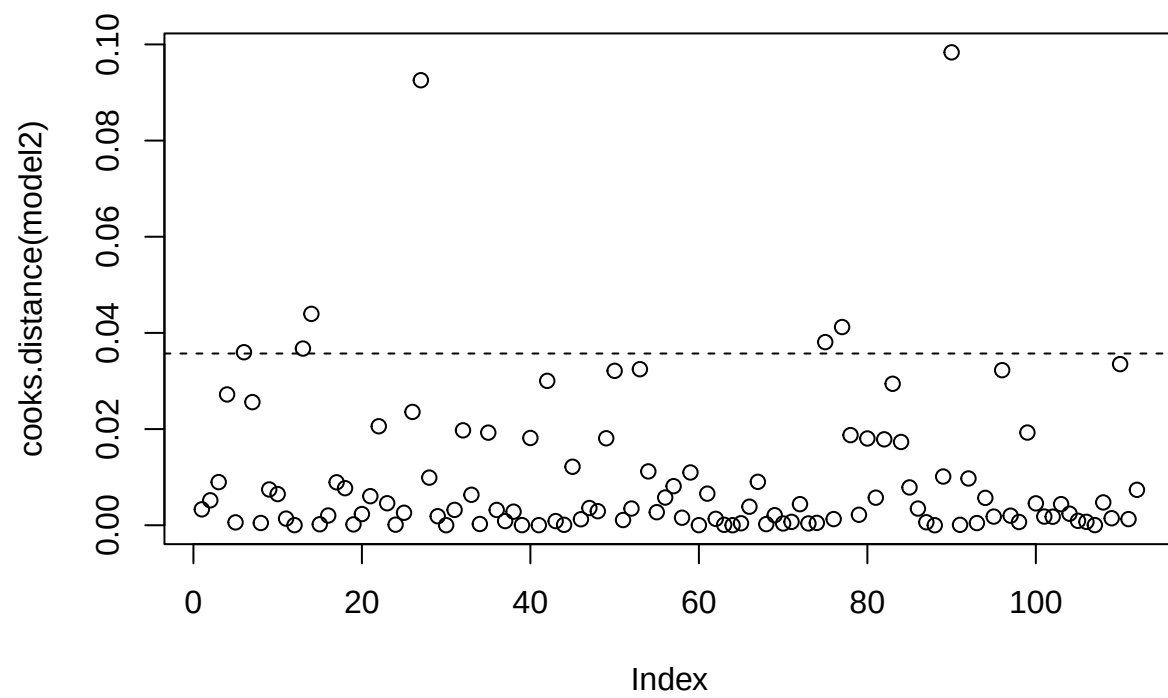


```
residuals_model <- residuals(model2)
ad.test(residuals_model)
```

```
##
## Anderson-Darling normality test
##
## data: residuals_model
## A = 0.16832, p-value = 0.9342
```

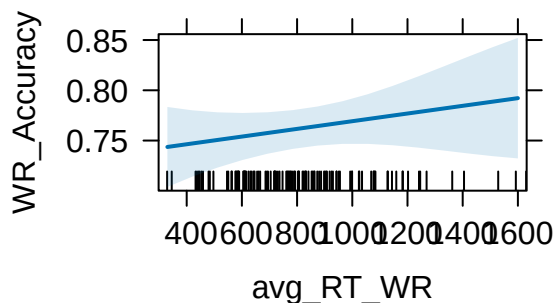
```
# Outliers
```

```
plot(cooks.distance(model2))
abline(h = 4/nrow(data), lty = 2)
```

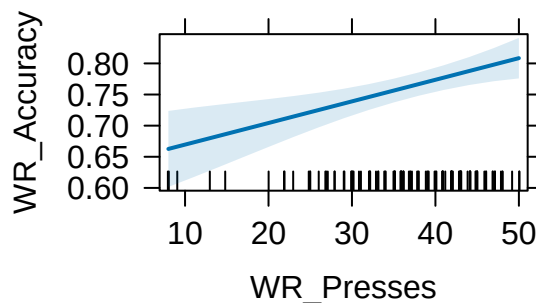


```
plot(allEffects(model2))
```

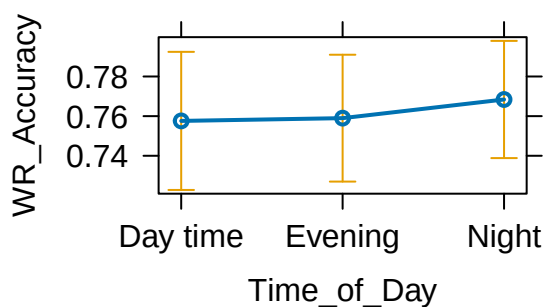
avg_RT_WR effect plot



WR_Presses effect plot



Time_of_Day effect plot



```
# Transform WR_Accuracy to (0,1) for beta regression because even though glm assumptions are satisfied,
n <- nrow(data)
any(data$WR_Accuracy <= 0 | data$WR_Accuracy >= 1, na.rm = TRUE)
```

```
## [1] TRUE
```

```
data <- data %>%
  mutate(
    WR_Accuracy_adj = (WR_Accuracy * (n - 1) + 0.5) / n
  )

model2 <- betareg(WR_Accuracy_adj ~ avg_RT_WR + WR_Presses + Time_of_Day, data = data)

# removing highly correlated predictors using VIF

removed_vars <- c()
repeat {
  vif_values <- vif(model2)
  threshold = 5

  if (is.matrix(vif_values)) {
    vif_values <- vif_values[, "GVIF^(1/(2*Df))"]
  }

  max_vif <- max(vif_values)
```

```

if (max_vif <= threshold) break

variable_to_remove <- names(which.max(vif_values))
removed_vars <- c(removed_vars, variable_to_remove)
formula_current <- formula(model2)
predictors <- attr(terms(formula_current), "term.labels")
predictors <- predictors[predictors != variable_to_remove]
response <- all.vars(formula_current)[1]
new_formula <- as.formula(
  paste(response, "~", paste(predictors, collapse = " + "))
)

model2 <- betareg(new_formula, data = data)
}

cat("Variables removed due to high VIF for model2:\n")

```

```
## Variables removed due to high VIF for model2:
```

```
print(removed_vars)
```

```
## NULL
```

```

# interpreting the model
cat("\nInterpretation\n")

```

```

##
## Interpretation

```

```
summary(model2)
```

```

##
## Call:
## betareg(formula = WR_Accuracy_adj ~ avg_RT_WR + WR_Presses + Time_of_Day,
##   data = data)
##
## Quantile residuals:
##      Min       1Q   Median       3Q      Max
## -1.8564 -0.6307 -0.1303  0.3842  4.2052
##
## Coefficients (mean model with logit link):
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    0.1750225  0.3039207   0.576 0.564695
## avg_RT_WR       0.0002526  0.0002222   1.137 0.255678
## WR_Presses      0.0198680  0.0059988   3.312 0.000926 ***
## Time_of_DayEvening 0.0766203  0.1457707   0.526 0.599151
## Time_of_DayNight  0.1053217  0.1385377   0.760 0.447112
##
## Phi coefficients (precision model with identity link):
##              Estimate Std. Error z value Pr(>|z|)

```

```
## (phi)    14.380      1.882    7.64 2.17e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Type of estimator: ML (maximum likelihood)
## Log-likelihood: 95.72 on 6 Df
## Pseudo R-squared: 0.07418
## Number of iterations: 21 (BFGS) + 2 (Fisher scoring)
```

```
confint(model2)
```

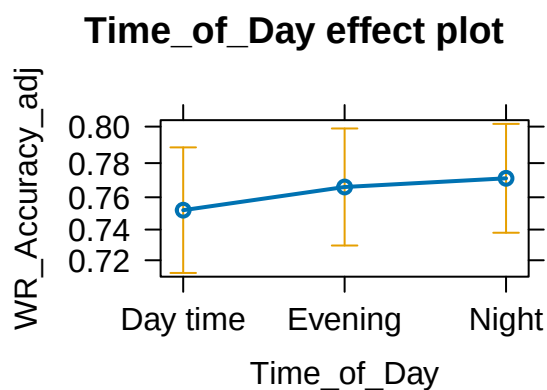
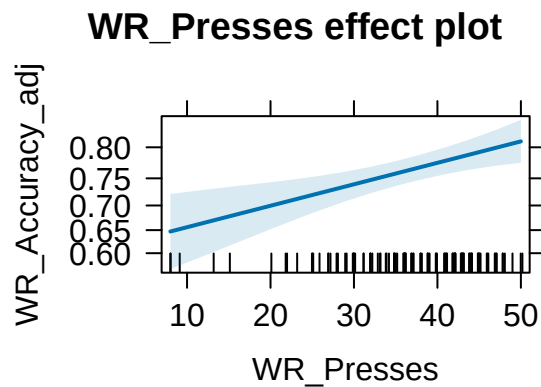
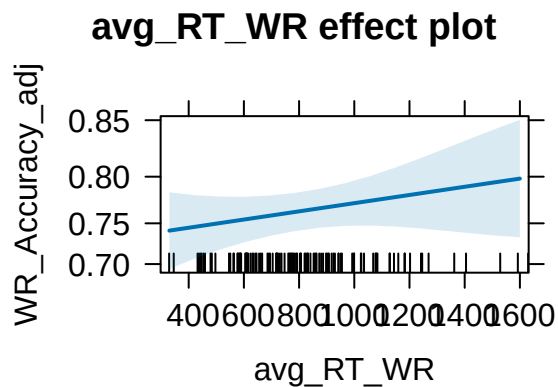
```
##              2.5 %      97.5 %
## (Intercept)   -0.4206511063 7.706962e-01
## avg_RT_WR     -0.0001829477 6.881117e-04
## WR_Presses    0.0081104837 3.162550e-02
## Time_of_DayEvening -0.2090850548 3.623256e-01
## Time_of_DayNight -0.1662072334 3.768506e-01
## (phi)         10.6908428064 1.806832e+01
```

```
avg_slopes(model2)
```

```
##
##      Term          Contrast Estimate Std. Error    z Pr(>|z|)    S
## Time_of_Day Evening - Day time 1.40e-02  2.66e-02 0.525    0.600 0.7
## Time_of_Day Night - Day time  1.91e-02  2.52e-02 0.757    0.449 1.2
## WR_Presses dY/dX              3.57e-03  1.07e-03 3.338    <0.001 10.2
## avg_RT_WR dY/dX              4.54e-05  3.99e-05 1.138    0.255 2.0
##      2.5 %  97.5 %
## -3.82e-02 0.066130
## -3.03e-02 0.068400
##  1.47e-03 0.005667
## -3.28e-05 0.000124
##
## Type: response
```

```
# Plotting the model
# The plot below shows how appropriate the model is for our data, and it's quite appropriate
# plot(model)
```

```
plot(allEffects(model2))
```



```
cat("AIC value of the model\n")
```

```
## AIC value of the model
```

```
AIC(model2)
```

```
## [1] -179.432
```

```
# Beta regression on sentence level - for each sentence, taking word condition, sentence voice and inter
```

```
data2 <- raw_dataset %>%
  filter(isTarget == TRUE) %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  mutate(
    # mapping conditions
    Word_Condition = case_when(
      str_detect(Stimulus, "^HH_") ~ "HH",
      str_detect(Stimulus, "^HVL_") ~ "HL",
      str_detect(Stimulus, "^LVH_") ~ "LH",
      str_detect(Stimulus, "^LVL_") ~ "LL"
    ),
    Word_Condition = factor(Word_Condition, levels = c("HH", "HL", "LH", "LL")),
    Sentence_Voice = case_when(
```



```

    str_detect(Stimulus, "_A$") ~ "Active",
    str_detect(Stimulus, "_P$") ~ "Passive",
    TRUE ~ NA_character_
  ),
  Sentence_Voice = factor(Sentence_Voice, levels = c("Active", "Passive")),

  Accuracy_IR = suppressWarnings(as.numeric(Accuracy_IR)),
  Accuracy_WR = suppressWarnings(as.numeric(Accuracy_WR)),

  # store previous IR result
  prev_IR = lag(Accuracy_IR)
) %>%
group_by(Stimulus, Word_Condition, Sentence_Voice) %>%
summarise(
  Participants = n_distinct(participant_ID),
  # IR components
  Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
  Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown"),
  Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy_IR == 1, na.rm = TRUE),
  False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

  # IR metrics
  Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
  False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
  IR_Memorability = Hit_Rate - False_Alarm_Rate,

  # WR components (after correct IR)
  WR_correct = sum(Event == "WR pressed" & Accuracy_WR == 1 & prev_IR == 1, na.rm = TRUE),
  WR_total = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),

  # WR metric
  WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),

  .groups = "drop"
) %>%
mutate(WR_Accuracy = replace_na(WR_Accuracy, 0))

data2 <- data2 %>%
  filter(Participants >= 15)

# beta regression for IR memorability

n <- nrow(data2)
any(data2$IR_Memorability <= 0 | data2$IR_Memorability >= 1, na.rm = TRUE)

## [1] TRUE

data2 <- data2 %>%
  mutate(
    IR_Memorability_adj = (IR_Memorability * (n - 1) + 0.5) / n
  )

model3 <- betareg(IR_Memorability_adj ~ Word_Condition*Sentence_Voice, data = data2)

```

```
# interpreting the model
cat("\nInterpretation\n")
```

```
##
## Interpretation
```

```
summary(model3)
```

```
##
## Call:
## betareg(formula = IR_Memorability_adj ~ Word_Condition * Sentence_Voice,
## data = data2)
##
## Quantile residuals:
##      Min       1Q   Median       3Q      Max
## -2.7220 -0.6527 -0.0921  0.3869  3.6008
##
## Coefficients (mean model with logit link):
##
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      1.34978    0.13004  10.379  <2e-16 ***
## Word_ConditionHL -0.08377    0.18319  -0.457  0.6475
## Word_ConditionLH -0.13000    0.18016  -0.722  0.4706
## Word_ConditionLL -0.25534    0.17744  -1.439  0.1501
## Sentence_VoicePassive -0.29967    0.17589  -1.704  0.0884 .
## Word_ConditionHL:Sentence_VoicePassive 0.27655    0.25285   1.094  0.2741
## Word_ConditionLH:Sentence_VoicePassive 0.26445    0.25006   1.058  0.2903
## Word_ConditionLL:Sentence_VoicePassive 0.40847    0.24833   1.645  0.1000 .
##
## Phi coefficients (precision model with identity link):
##      Estimate Std. Error z value Pr(>|z|)
## (phi)   6.9885    0.5405  12.93  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Type of estimator: ML (maximum likelihood)
## Log-likelihood: 187.7 on 9 Df
## Pseudo R-squared: 0.009057
## Number of iterations: 19 (BFGS) + 2 (Fisher scoring)
```

```
confint(model3)
```

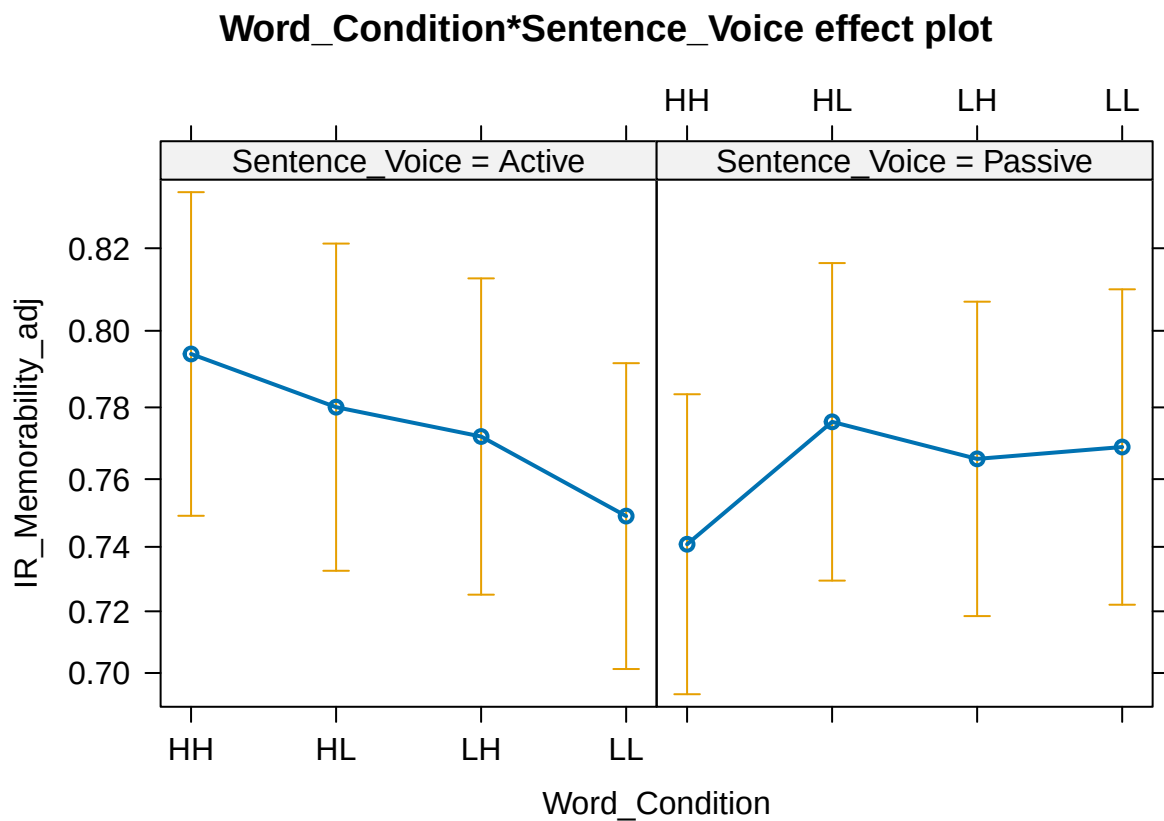
```
##              2.5 %      97.5 %
## (Intercept)   1.09489881 1.60466233
## Word_ConditionHL -0.44282462 0.27528155
## Word_ConditionLH -0.48309658 0.22310414
## Word_ConditionLL -0.60311711 0.09243781
## Sentence_VoicePassive -0.64441340 0.04506949
## Word_ConditionHL:Sentence_VoicePassive -0.21901457 0.77212406
## Word_ConditionLH:Sentence_VoicePassive -0.22566704 0.75455979
## Word_ConditionLL:Sentence_VoicePassive -0.07824075 0.89517896
## (phi)         5.92907831 8.04794816
```

```
avg_slopes(model3)
```

```
##
##          Term          Contrast Estimate Std. Error      z Pr(>|z|)    S
## Sentence_Voice Passive - Active -0.01113    0.0158 -0.7060    0.480 1.1
## Word_Condition HL - HH          0.01085    0.0221  0.4907    0.624 0.7
## Word_Condition LH - HH          0.00168    0.0222  0.0755    0.940 0.1
## Word_Condition LL - HH         -0.00794    0.0223 -0.3551    0.723 0.5
##   2.5 % 97.5 %
## -0.0420 0.0198
## -0.0325 0.0542
## -0.0418 0.0452
## -0.0517 0.0359
##
## Type: response
```

```
# Plotting the model
# The plot below shows how appropriate the model is for our data, and it's quite appropriate
# plot(model)
```

```
plot(allEffects(model3))
```



```
cat("AIC value of the model\n")
```

```
## AIC value of the model
```

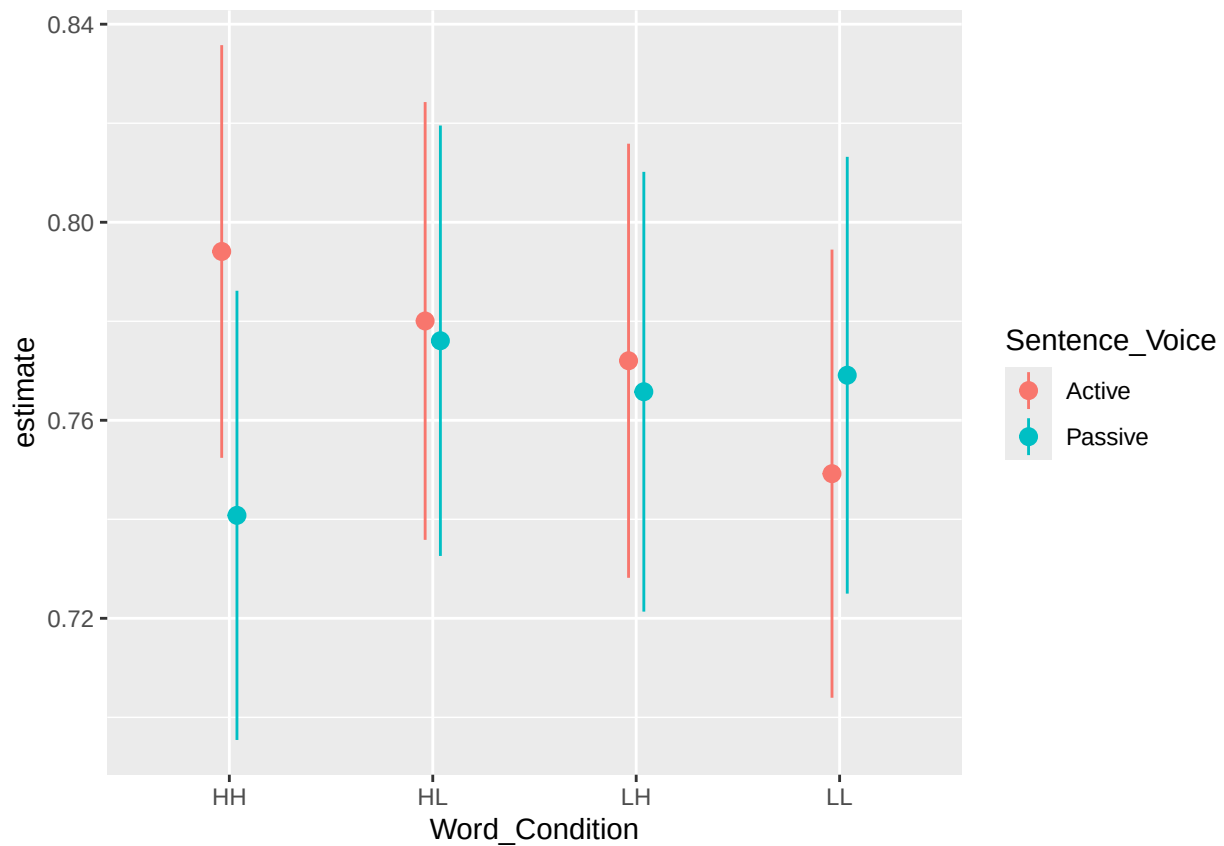
```
AIC(model3)
```

```
## [1] -357.3176
```

```
cat("Plotting Predictions\n")
```

```
## Plotting Predictions
```

```
plot_predictions(model3, condition = c("Word_Condition", "Sentence_Voice"))
```



```
# beta regression for WR_Accuracy
```

```
n <- nrow(data2)
```

```
any(data2$WR_Accuracy <= 0 | data2$WR_Accuracy >= 1, na.rm = TRUE)
```

```
## [1] TRUE
```

```
data2 <- data2 %>%
  mutate(
    WR_Accuracy_adj = (WR_Accuracy * (n - 1) + 0.5) / n
  )

model4 <- betareg(WR_Accuracy_adj ~ Word_Condition*Sentence_Voice, data = data2)

# interpreting the model
cat("\nInterpretation\n")
```

```
##
## Interpretation
```

```
summary(model4)
```

```
##
## Call:
## betareg(formula = WR_Accuracy_adj ~ Word_Condition * Sentence_Voice,
## data = data2)
##
## Quantile residuals:
##      Min       1Q   Median       3Q      Max
## -1.9273 -0.6885 -0.2014  0.2890  2.9487
##
## Coefficients (mean model with logit link):
##
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      1.245633   0.150112   8.298  <2e-16
## Word_ConditionHL    0.103737   0.213197   0.487  0.6266
## Word_ConditionLH    0.440651   0.213976   2.059  0.0395
## Word_ConditionLL   -0.001020   0.207937  -0.005  0.9961
## Sentence_VoicePassive -0.007441   0.206618  -0.036  0.9713
## Word_ConditionHL:Sentence_VoicePassive -0.121855   0.296710  -0.411  0.6813
## Word_ConditionLH:Sentence_VoicePassive -0.412804   0.297576  -1.387  0.1654
## Word_ConditionLL:Sentence_VoicePassive -0.280360   0.291174  -0.963  0.3356
##
## (Intercept)          ***
## Word_ConditionHL
## Word_ConditionLH          *
## Word_ConditionLL
## Sentence_VoicePassive
## Word_ConditionHL:Sentence_VoicePassive
## Word_ConditionLH:Sentence_VoicePassive
## Word_ConditionLL:Sentence_VoicePassive
##
## Phi coefficients (precision model with identity link):
##      Estimate Std. Error z value Pr(>|z|)
## (phi)   4.3373    0.3369  12.87  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Type of estimator: ML (maximum likelihood)
```

```
## Log-likelihood: 176.4 on 9 Df
## Pseudo R-squared: 0.02848
## Number of iterations: 26 (BFGS) + 2 (Fisher scoring)
```

```
confint(model4)
```

```
##              2.5 %    97.5 %
## (Intercept)      0.9514192 1.5398464
## Word_ConditionHL -0.3141222 0.5215955
## Word_ConditionLH  0.0212657 0.8600362
## Word_ConditionLL -0.4085694 0.4065298
## Sentence_VoicePassive -0.4124052 0.3975235
## Word_ConditionHL:Sentence_VoicePassive -0.7033950 0.4596855
## Word_ConditionLH:Sentence_VoicePassive -0.9960418 0.1704328
## Word_ConditionLL:Sentence_VoicePassive -0.8510504 0.2903310
## (phi)            3.6769382 4.9976864
```

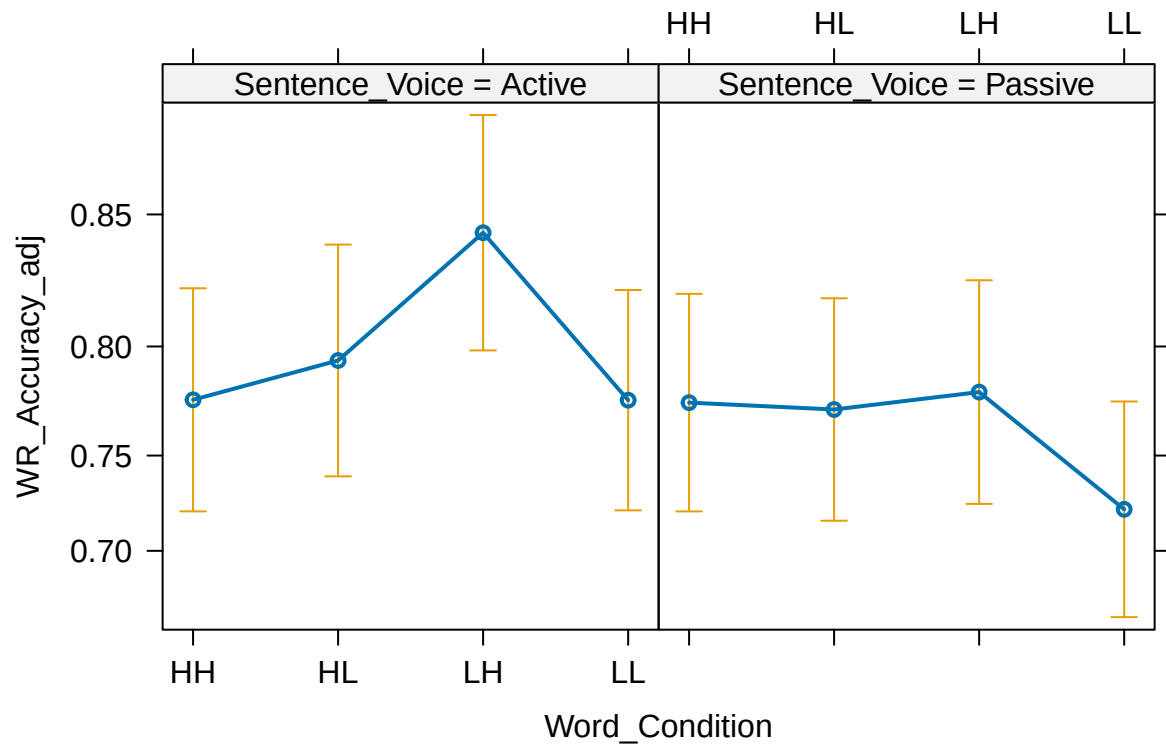
```
avg_slopes(model4)
```

```
##
##      Term      Contrast Estimate Std. Error      z Pr(>|z|)      S
## Sentence_Voice Passive - Active -0.03513      0.0179 -1.960      0.050 4.3
## Word_Condition HL - HH      0.00706      0.0255  0.277      0.782 0.4
## Word_Condition LH - HH      0.03570      0.0243  1.471      0.141 2.8
## Word_Condition LL - HH     -0.02672      0.0263 -1.015      0.310 1.7
##      2.5 %    97.5 %
## -0.0703 -1.66e-06
## -0.0429  5.70e-02
## -0.0119  8.33e-02
## -0.0783  2.49e-02
##
## Type: response
```

```
# Plotting the model
# The plot below shows how appropriate the model is for our data, and it's quite appropriate
# plot(model)
```

```
plot(allEffects(model4))
```

Word_Condition*Sentence_Voice effect plot



```
cat("AIC value of the model\n")
```

```
## AIC value of the model
```

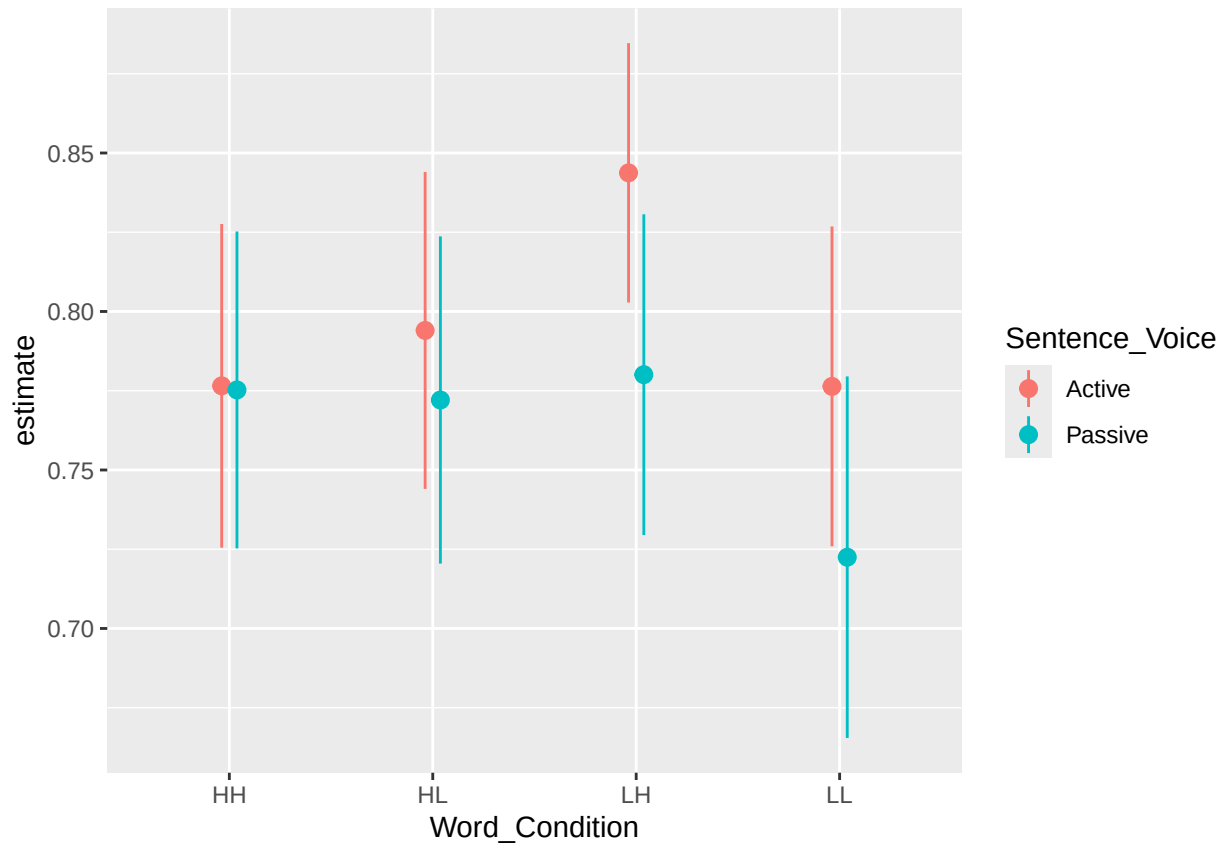
```
AIC(model4)
```

```
## [1] -334.7383
```

```
cat("Plotting Predictions\n")
```

```
## Plotting Predictions
```

```
plot_predictions(model4, condition = c("Word_Condition", "Sentence_Voice"))
```



```
#
# library(tidyverse)
# library(stringr)
# library(lme4)
# library(marginaleffects)
#
# raw_dataset <- read_csv(
#   "sentence_memorability_data.csv",
#   na = c("", "NA", "N/A"),
#   show_col_types = FALSE
# )
#
# trial_data <- raw_dataset %>%
#   filter(isTarget == TRUE) %>%
#   arrange(participant_ID, Stimulus, Timestamp) %>%
#   group_by(participant_ID, Stimulus) %>%
#   mutate(
#     # Word condition
#     Word_Condition = case_when(
#       str_detect(Stimulus, "^HH_") ~ "HH",
#       str_detect(Stimulus, "^HVL_") ~ "HL",
#       str_detect(Stimulus, "^LVH_") ~ "LH",
#       str_detect(Stimulus, "^LVL_") ~ "LL"
```



```

#   ),
#   Word_Condition = factor(Word_Condition, levels = c("HH", "HL", "LH", "LL")),
#
#   # Sentence voice
#   Sentence_Voice = case_when(
#     str_detect(Stimulus, "_A$") ~ "Active",
#     str_detect(Stimulus, "_P$") ~ "Passive"
#   ),
#   Sentence_Voice = factor(Sentence_Voice, levels = c("Active", "Passive")),
#
#   # Convert accuracy
#   Accuracy.IR = as.numeric(Accuracy.IR),
#   Accuracy.WR = as.numeric(Accuracy.WR),
#
#   # Lag IR (needed for WR)
#   prev_IR = lag(Accuracy.IR)
# ) %>%
#   ungroup()
#
# ir_responses <- trial_data %>%
#   filter(Event == "IR pressed") %>%
#   select(
#     participant_ID,
#     Stimulus,
#     Accuracy.IR
#   )
#
# sentence_trials <- trial_data %>%
#   filter(Event == "Sentence shown") %>%
#   select(
#     participant_ID,
#     Stimulus,
#     isRepeat,
#     Word_Condition,
#     Sentence_Voice
#   )
#
# ir_data <- sentence_trials %>%
#   left_join(ir_responses,
#     by = c("participant_ID", "Stimulus")) %>%
#   mutate(
#     IR_correct = case_when(
#       isRepeat == TRUE & Accuracy.IR == 1 ~ 1, # hit
#       isRepeat == TRUE & is.na(Accuracy.IR) ~ 0, # miss (no press)
#       is.na(isRepeat) & Accuracy.IR == 1 ~ 0, # false alarm
#       is.na(isRepeat) & is.na(Accuracy.IR) ~ 1 # correct rejection
#     )
#   ) %>%
#   mutate(Stimulus = factor(Stimulus))
# # Check variation (important debug step)
# table(ir_data$IR_correct)
#
# ir_data <- ir_data %>%

```

```

#   mutate(
#     Stimulus = as.factor(Stimulus)
#   )
#
# model_ir <- glmer(
#   IR_correct ~ Word_Condition * Sentence_Voice +
#     (1 | Stimulus),
#   data = ir_data,
#   family = binomial,
#   control = glmerControl(optimizer = "bobyqa")
# )
#
# summary(model_ir)
#
#
# wr_data <- trial_data %>%
#   filter(Event == "WR pressed", prev_IR == 1) %>%
#   mutate(
#     WR_correct = if_else(Accuracy.WR == 1, 1, 0)
#   )
#
# # Check variation
# table(wr_data$WR_correct)
#
# wr_data <- wr_data %>%
#   mutate(
#     Stimulus = as.factor(Stimulus)
#   )
#
# model_wr <- glmer(
#   WR_correct ~ Word_Condition * Sentence_Voice +
#     (1 | Stimulus),
#   data = wr_data,
#   family = binomial,
#   control = glmerControl(optimizer = "bobyqa")
# )
#
# summary(model_wr)
#
# # IR predictions
# plot_predictions(
#   model_ir,
#   condition = c("Word_Condition", "Sentence_Voice")
# ) +
#   labs(
#     title = "IR Mixed Model Predictions",
#     y = "Probability of Correct Recognition"
#   )
#
# # WR predictions
# plot_predictions(
#   model_wr,
#   condition = c("Word_Condition", "Sentence_Voice")

```

```
# ) +  
#   labs(  
#     title = "WR Mixed Model Predictions",  
#     y = "Probability of Correct Recall"  
#   )  
#
```